

Best Practices Guide

Comprehensive best practices for VM migrations with Hyper2KVM based on real-world experience and lessons learned.

Quick Links

- [General Best Practices](#)
 - [Security Best Practices](#)
 - [Performance Best Practices](#)
 - [Reliability Best Practices](#)
 - [Cost Optimization](#)
 - [Team & Process Best Practices](#)
 - [Common Anti-Patterns to Avoid](#)
-

General Best Practices

1. Always Test First

DO:

```
# Test with non-critical VM first
command: local
vmdk: /vms/test-dev-server.vmdk
output_dir: /kvm/test
to_output: test-server.qcow2

# Enable testing
libvirt_test: true
```

DON'T: - Migrate production VMs without testing - Skip the test migration phase - Assume everything will work first time

Why: Testing identifies issues before they impact production.

2. Always Use Essential Features

DO:

```
# Essential options for Linux VMs
fstab_mode: stabilize-all
regen_initramfs: true
# grub is auto-handled

# Essential for Windows VMs
inject_virtio_drivers: true
windows_version: 2019
```

DON'T:

```
# Missing critical options
fstab_mode: preserve # Will likely cause boot failures
regen_initramfs: false # Missing VirtIO drivers
```

Why: These options prevent 90% of boot failures.

3. Inspect Before Migrating

DO:

```
# Always inspect VMDK first
./scripts/vmdk_inspect.py /vms/production.vmdk > inspection-report.txt

# Review the report
cat inspection-report.txt

# Check for issues
grep -i "issue\|warning\|error" inspection-report.txt
```

DON'T: - Skip VMDK inspection - Ignore inspection warnings - Assume VMDK is healthy

Why: Early detection of issues saves time and prevents failures.

4. Keep Backups Until Verified

DO:

```
# Keep originals by default
keep_original: true
```

```
# Don't delete source VMs immediately
# Keep for 1-2 weeks after successful migration
```

DON'T: - Delete source VMs immediately after migration - Assume migration is successful without validation - Skip backup verification

Why: Easy rollback if issues are discovered later.

5. Document Everything

DO: - Use the [Migration Runbook Template](#) - Document configuration for each VM - Keep migration logs - Record lessons learned

DON'T: - Rely on memory - Skip documentation - Delete logs after migration

Why: Documentation helps troubleshooting and future migrations.

Security Best Practices

1. Use SSH Keys, Not Passwords

DO:

```
# Remote fetch with SSH key
command: fetch-and-fix
host: esxi-01.example.com
user: root
identity: ~/.ssh/migration_key # Dedicated key
remote: /vmfs/volumes/datastore1/vm.vmdk
output_dir: /kvm/vms
```

```
# Generate dedicated key for migrations
ssh-keygen -t rsa -b 4096 -f ~/.ssh/migration_key -C "migration-key"

# Use strong passphrase
# Store passphrase in secure vault
```

DON'T:

```
# Don't store passwords in configs
vcenter_password: "MyPassword123" # BAD!
```

Why: SSH keys are more secure and auditable.

2. Use Least Privilege

DO: - Create dedicated migration user accounts - Grant only necessary permissions - Use read-only access when possible - Revoke access after migration

DON'T: - Use root/administrator for everything - Grant broad permissions - Keep migration accounts active indefinitely

Why: Limits potential damage from compromised credentials.

3. Encrypt Sensitive Data

DO:

```
# Encrypt QCOW2 images if they contain sensitive data
qemu-img convert -O qcow2 -o encrypt.format=luks,encrypt.key-secret=sec0 \
    input.qcow2 encrypted-output.qcow2

# Use encrypted channels for remote transfers
# SSH, HTTPS, VPN
```

DON'T: - Transfer unencrypted VMs over untrusted networks - Store sensitive VMs on unencrypted storage

Why: Protects data in transit and at rest.

4. Audit and Log Everything

DO:

```
# Enable detailed logging
log_level: INFO
log_file: /var/log/hyper2kvm/migration-${VM_NAME}.log

# Keep logs for compliance
# Archive logs for audit trail
```

```
# Review logs after migration
grep -i "error\|warning" /var/log/hyper2kvm/*.log
```

DON'T: - Disable logging - Delete logs immediately - Ignore security events in logs

Why: Logs provide audit trail and help with security investigations.

5. Validate Security Posture After Migration

DO:

```
# After migration, verify security
# Check firewall rules
iptables -L -n

# Check SELinux status
getenforce

# Check open ports
ss -tlnp

# Verify SSH configuration
sshd -T | grep -i permit
```

DON'T: - Assume security settings migrated correctly - Skip security validation

Why: Migration may change security configurations.

Performance Best Practices

1. Choose the Right Output Format

DO:

```
# For databases and high-I/O workloads
out_format: raw
compress: false

# For general workloads
out_format: qcow2
compress: false

# For storage-constrained environments
out_format: qcow2
compress: true
compression_level: 6
```

DON'T:

```
# Don't compress database VMs
out_format: qcow2
compress: true # Adds CPU overhead
```

Why: Format impacts runtime performance.

Performance Comparison: - Raw: 100% I/O performance - QCOW2: 95% I/O performance - QCOW2 compressed: 95% I/O performance (no runtime penalty)

2. Use Appropriate Storage

DO: - Use SSD/NVMe for migration host temp space - Use fast storage for output directory - Consider network-attached storage carefully

```
# Use fast local storage for conversion
conversion_dir: /fast/ssd/temp
output_dir: /fast/storage/vms
```

DON'T: - Use slow HDD for temp space - Use network storage for conversion_dir -
Ignore I/O bottlenecks

Why: Migration speed is often I/O-bound.

3. Optimize Parallel Migrations

DO:

```
# Batch with appropriate parallelism
batch_parallel: 3 # For 4-core system

# Consider system resources
# CPUs: 4 → parallel: 2-3
# CPUs: 8 → parallel: 4-6
# CPUs: 16 → parallel: 8-12
```

DON'T:

```
# Too many parallel migrations
batch_parallel: 10 # On 4-core system - BAD!
```

Why: Over-parallelization causes resource contention.

Rule of Thumb: parallel = (CPU_count / 2) to (CPU_count - 1)

4. Monitor Resource Usage

DO:

```
# Monitor during migration
watch -n 2 'ps aux | grep hyper2kvm; iostat -x 1 1; free -h'

# Check for bottlenecks
# CPU: Should be high (80-95%)
# I/O wait: Should be low (<20%)
# Memory: Should have 2+ GB available
```

DON'T: - Run migrations without monitoring - Ignore resource warnings - Starve other critical services

Why: Monitoring identifies bottlenecks and prevents issues.

5. Optimize Network Transfers

DO:

```
# For remote migrations
command: fetch-and-fix
host: esxi.example.com
user: root
identity: ~/.ssh/id_rsa

# Use compression for slow networks
compress: true

# Adjust timeouts for slow networks
timeout: 7200
network_retry: 5
```

```
# Test network speed first
iperf3 -c esxi.example.com
```

DON'T: - Use compression on fast networks (>1 Gbps) - Ignore network bottlenecks - Use default timeouts on slow networks

Why: Network is often the bottleneck for remote migrations.

Reliability Best Practices

1. Use Idempotent Operations

DO:

```
# Use configs that can be re-run
h2kvmctl --config migration.yaml
```



```
# If migration fails, fix issue and re-run same command
# Hyper2KVM is designed to be idempotent
```

DON'T: - Create one-off commands that can't be repeated - Manually modify files during migration - Skip configuration management

Why: Makes recovery from failures easier.

2. Implement Proper Error Handling

DO:

```
# Check exit codes
if h2kvmctl --config migration.yaml; then
    echo "Migration successful"
    # Continue with validation
else
    echo "Migration failed"
    # Alert team
    # Review logs
    # Don't proceed to cutover
fi
```

DON'T:

```
# Ignore errors
h2kvmctl --config migration.yaml || true # BAD!
```

Why: Proper error handling prevents cascading failures.

3. Use Staged Rollouts

DO:

```
Migration Phases:
1. Test/Dev VMs (validate process)
```

- 2. Non-critical production (validate at scale)
- 3. Critical production (apply lessons learned)

DON'T: - Migrate all VMs at once - Start with most critical VMs - Skip testing phases

Why: Staged rollouts reduce risk and allow for process improvements.

4. Maintain Rollback Capability

DO: - Keep source VMs running during validation - Document rollback procedure - Test rollback before migration - Set rollback decision points

DON'T: - Decommission source VMs immediately - Skip rollback planning - Assume migrations always succeed

Why: Rollback capability is your safety net.

5. Validate Thoroughly

DO:

```
# Multi-level validation
# 1. File integrity
qemu-img check output.qcow2

# 2. Boot test
virsh start vm-name

# 3. Application test
curl http://vm-name/health

# 4. Performance test
run-benchmark.sh

# 5. Integration test
test-dependencies.sh
```

DON'T: - Skip validation steps - Assume boot success means everything works - Rush through validation

Why: Thorough validation catches issues before users do.

Cost Optimization Best Practices

1. Right-Size Resources

DO: - Review VM resource allocation before migration - Right-size CPU and memory - Remove unused VMs before migrating

```
# Analyze VM resource usage before migration
# Right-size based on actual usage, not allocated resources
```

DON'T: - Migrate oversized VMs without review - Keep the same resource allocation automatically - Migrate unused VMs

Why: Migration is an opportunity to optimize costs.

2. Use Compression Wisely

DO:

```
# Use compression for archival/cold storage
out_format: qcow2
compress: true
compression_level: 9 # Maximum compression
```

DON'T:

```
# Don't compress if storage is cheap and abundant
compress: true # Wastes CPU time during migration
```

Why: Compression trades CPU time for storage savings.

When to Compress: - Storage is expensive or limited - VMs are infrequently accessed - Migration time is not critical - Fast migration is priority - Storage is cheap and abundant

3. Batch Efficiently

DO:

```
# Batch similar VMs together
# Migrate during off-hours to use spare capacity
batch_parallel: 4
```

DON'T: - Migrate VMs one at a time when batch is possible - Waste idle compute resources

Why: Batching maximizes resource utilization.

4. Choose Cost-Effective Storage

DO: - Use appropriate storage tier for each VM - High-performance VMs → SSD/NVMe
- Archive VMs → Cheaper HDD - Consider cloud storage tiers

DON'T: - Put all VMs on most expensive storage - Ignore storage costs

Why: Storage tier should match VM requirements and SLA.

Team & Process Best Practices

1. Define Clear Roles

DO:

```
Migration Team Roles:
- Migration Lead: Overall coordination
- Technical Lead: Technical decisions
- Migration Engineer: Execute migrations
- Application Owner: Validate applications
- Operations: Post-migration support
```

DON'T: - Leave roles undefined - Have one person do everything - Skip stakeholder involvement

Why: Clear roles prevent confusion and ensure accountability.

2. Use Runbooks

DO: - Create runbook for each migration project - Review runbook with team before execution - Update runbook with actual times and lessons learned - Archive runbooks for future reference

Template: [Migration Runbook Template](#)

DON'T: - Wing it without documentation - Rely on tribal knowledge - Skip post-migration documentation

Why: Runbooks ensure consistency and knowledge transfer.

3. Communicate Proactively

DO:

Communication Schedule:

- T-1 week: Initial notification
- T-3 days: Reminder with details
- T-1 day: Final confirmation
- T-0: Start notification
- T+0: Completion notification
- T+1 day: Status update

DON'T: - Surprise users with migrations - Communicate only when there are problems - Skip status updates

Why: Proactive communication reduces user impact and builds trust.

4. Learn from Each Migration

DO: - Hold post-migration retrospective - Document lessons learned - Update processes based on learnings - Share knowledge with team

Retrospective Questions: - What went well? - What could be improved? - What did we learn? - What will we do differently next time?

DON'T: - Skip retrospectives - Repeat the same mistakes - Keep knowledge siloed

Why: Continuous improvement increases success rate.

5. Maintain Documentation

DO: - Keep documentation current - Update after each migration - Version control configurations - Archive completed migrations

DON'T: - Let documentation become stale - Rely on outdated procedures - Delete migration records

Why: Good documentation accelerates future migrations.

Common Anti-Patterns to Avoid

Anti-Pattern 1: "Big Bang" Migration

What: Migrating all VMs at once without testing.

Why it's bad: - High risk - No opportunity to learn - Difficult to troubleshoot - Hard to rollback

Instead: - Staged rollout (test → dev → non-critical → critical) - Learn from each phase - Adjust process based on learnings

Anti-Pattern 2: Skipping Pre-Flight Checks

What: Starting migration without validation.

Why it's bad: - Wastes time on preventable failures - Discovers issues too late - Increases stress during migration

Instead: - Run [Pre-Flight Validation](#) - Fix issues before migration day - Validate test migrations

Anti-Pattern 3: Ignoring Inspection Warnings

What: Proceeding despite VMDK inspection warnings.

Why it's bad: - Warnings often indicate real problems - Issues manifest during or after migration - Harder to fix after migration

Instead: - Address all warnings before migration - Fix issues on source VMs when possible - Use appropriate migration options (e.g., `xfs_regenerate_uuid`)

Anti-Pattern 4: No Rollback Plan

What: Migrating without a way to rollback.

Why it's bad: - No safety net if issues arise - Pressure to make broken migration work - Extended downtime if migration fails

Instead: - Plan rollback before migration - Keep source VMs available - Test rollback procedure - Define rollback decision points

Anti-Pattern 5: Inadequate Testing

What: Superficial or no testing before production.

Why it's bad: - Issues discovered in production - User impact - Loss of confidence

Instead: - Test with similar VMs first - Validate boot, network, applications - Performance test - Get user acceptance

Anti-Pattern 6: Manual Configuration

What: Manually modifying VMs instead of using automation.

Why it's bad: - Not repeatable - Human error - Difficult to troubleshoot - Can't rollback easily

Instead: - Use YAML configurations - Version control configs - Automate everything possible - Document manual steps clearly

Anti-Pattern 7: Premature Decommission

What: Deleting source VMs immediately after migration.

Why it's bad: - No rollback capability - Issues may appear later - Data loss risk

Instead: - Keep source VMs for 1-2 weeks minimum - Validate thoroughly before decommissioning - Get stakeholder sign-off - Archive source VMs if needed

Anti-Pattern 8: Ignoring Security

What: Treating security as an afterthought.

Why it's bad: - Security vulnerabilities - Compliance violations - Data exposure

Instead: - Plan security from the start - Follow [Security Best Practices](#) - Validate security posture after migration - Audit and log everything

Anti-Pattern 9: Poor Communication

What: Not communicating with stakeholders.

Why it's bad: - Surprised users - Lack of support when issues arise - Blame when things go wrong

Instead: - Communicate early and often - Set clear expectations - Provide status updates - Involve stakeholders in validation

Anti-Pattern 10: Skipping Documentation

What: Not documenting migrations.

Why it's bad: - Can't troubleshoot effectively - Can't repeat successful migrations - Knowledge loss when team members leave - Compliance issues

Instead: - Use [Migration Runbook Template](#) - Document configurations - Keep logs - Archive completed migrations

Quick Reference: Best Practices Checklist

Before Migration

- ☐ Run pre-flight validation
- ☐ Inspect all VMDKs
- ☐ Test with non-critical VM
- ☐ Create runbook
- ☐ Review with team
- ☐ Communicate with stakeholders
- ☐ Verify backups

During Migration

- ☐ Follow runbook
- ☐ Use essential features (fstab_mode, regen_initramfs, etc.)
- ☐ Monitor resources
- ☐ Log everything
- ☐ Validate each VM
- ☐ Update runbook with actuals

After Migration

- ☐ Thorough validation (boot, network, apps, performance)
- ☐

- ☐ Get stakeholder sign-off
- ☐ Keep source VMs for rollback window
- ☐ Monitor migrated VMs
- ☐ Document lessons learned
- ☐ Update documentation

Success Metrics

Track these metrics to measure migration success:

Metric	Target	Good	Acceptable
Success Rate	100%	>95%	>90%
First Boot Success	100%	>97%	>95%
Rollback Rate	0%	<3%	<5%
Actual vs Planned Time	100%	>90%	>80%
Issues Found in Production	0	<2%	<5%
Stakeholder Satisfaction	100%	>90%	>80%

Additional Resources

- [Migration Checklist](#) - Comprehensive checklists
- [Pre-Flight Validation](#) - Validation guide
- [Migration Runbook Template](#) - Runbook template
- [Security Best Practices](#) - Security guide
- [Troubleshooting Flowchart](#) - Problem solving

Last Updated: February 2026 **Documentation Version:** 2.1.0